

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Assignment 16: Develop a multithreaded C program using POSIX threads where multiple threads coordinate access to a shared resource using either semaphores or condition variables. You may implement a simple producer-consumer system, limited resource access system, or thread scheduling simulation. The program should ensure that threads wait correctly when the resource is unavailable and continue execution only when signaled.

Demonstrate proper synchronization, safe shared-memory access, and thread communication using functions such as `sem_wait()`, `sem_post()`, `pthread_cond_wait()`, or `pthread_cond_signal()`. Print messages showing thread execution order and explain how synchronization prevents inconsistent behavior.

Answer:

// Assignment 16

// Producer-Consumer using POSIX Threads and Semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 5
#define ITEMS 10
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0;  
int out = 0;
```

```
sem_t empty;  
sem_t full;  
pthread_mutex_t mutex;
```

```
void *producer(void *arg)  
{  
    int i;  
  
    for(i = 1; i <= ITEMS; i++)  
    {  
        sem_wait(&empty);  
  
        pthread_mutex_lock(&mutex);  
  
        buffer[in] = i;  
        printf("Producer produced item %d at position %d\n", i, in);  
  
        in = (in + 1) % BUFFER_SIZE;  
  
        pthread_mutex_unlock(&mutex);  
  
        sem_post(&full);  
  
        sleep(1);  
    }  
  
    pthread_exit(NULL);  
}
```

```

void *consumer(void *arg)
{
    int item;
    int i;

    for(i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed item %d from position %d\n", item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);
    }

    pthread_exit(NULL);
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

```

```
sem_init(&empty, 0, BUFFER_SIZE);
```

```
sem_init(&full, 0, 0);
```

```
pthread_mutex_init(&mutex, NULL);
```

```
pthread_create(&producer_thread, NULL, producer, NULL);
```

```
pthread_create(&consumer_thread, NULL, consumer, NULL);
```

```
pthread_join(producer_thread, NULL);
```

```
pthread_join(consumer_thread, NULL);
```

```
sem_destroy(&empty);
```

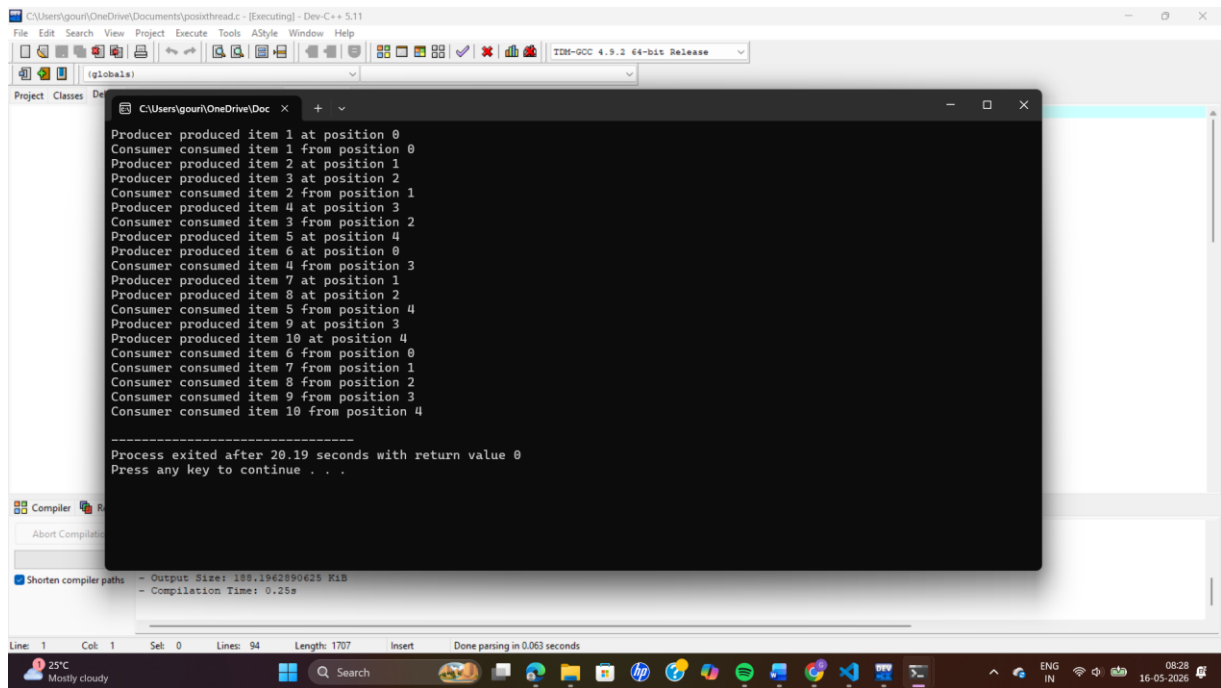
```
sem_destroy(&full);
```

```
pthread_mutex_destroy(&mutex);
```

```
return 0;
```

```
}
```

Screenshots of the output:



```
C:\Users\gauri\OneDrive\Documents\posixthread.c - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes De
C:\Users\gauri\OneDrive\Doc
Producer produced item 1 at position 0
Consumer consumed item 1 from position 0
Producer produced item 2 at position 1
Producer produced item 3 at position 2
Consumer consumed item 2 from position 1
Producer produced item 4 at position 3
Consumer consumed item 3 from position 2
Producer produced item 5 at position 4
Producer produced item 6 at position 0
Consumer consumed item 4 from position 3
Producer produced item 7 at position 1
Producer produced item 8 at position 2
Consumer consumed item 5 from position 4
Producer produced item 9 at position 3
Producer produced item 10 at position 4
Consumer consumed item 6 from position 0
Consumer consumed item 7 from position 1
Consumer consumed item 8 from position 2
Consumer consumed item 9 from position 3
Consumer consumed item 10 from position 4

-----
Process exited after 20.19 seconds with return value 0
Press any key to continue . . .

Compiler
Abort Compilation
Shorten compiler paths - Output Size: 189,194,289,0425 KIB
- Compilation Time: 0.25s
Line: 1 Col: 1 Sel: 0 Lines: 94 Length: 1707 Insert Done parsing in 0.063 seconds
25°C Mostly cloudy
Search
ENG IN
08:28 16-05-2026
```